tidy3d_101

August 6, 2026

1 Getting Started with tidy3d

This notebook introduces **tidy3d** to new users at Academia Sinica. It covers the basics needed to run a first simulation and understand what tidy3d does internally.

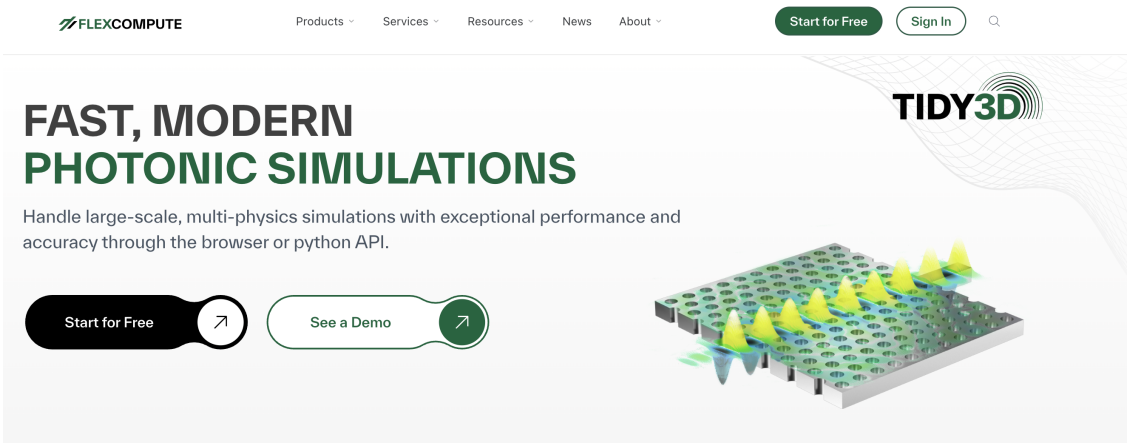
1.1 What is tidy3d?

tidy3d is a software package by Flexcompute for simulating electromagnetic fields, mainly for nanophotonics problems. It solves Maxwell's equations using the finite-difference time-domain (FDTD) method. Simulations run on Flexcompute's cloud servers, so even large 3D models typically finish in minutes.

1.2 A brief word on FDTD

An FDTD simulation numerically solves Maxwell's equations through these steps:

- Define a computational domain, a grid resolution, and boundary conditions. Boundary conditions may be absorbing or periodic.
- Place material bodies with specified optical properties, such as permittivity and permeability, inside the domain.
- Define a source.
- The source generates a time-limited electromagnetic pulse. Its spectral content should cover the frequency range of interest.
- Record the fields over time and apply a Fourier transform to obtain their frequency-domain representation.

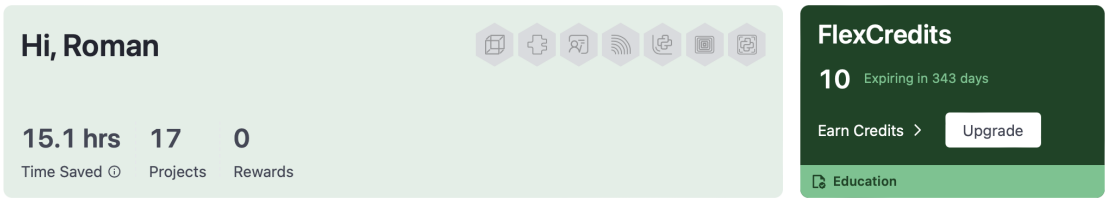


1.3 Registration

Create a free account at tidy3d.simulation.cloud/signup. New accounts start with 10 trial FlexCredits, enough to run several small test simulations.

Afterwards, you can confirm your student status by verifying with your university email address. This unlocks 10 FlexCredits every month.

```
[ ]: from IPython.display import Image, display

display(Image(filename="/content/images/dashboard.png"))
```



1.4 Working with tidy3d: UI or Python

Tidy3D includes a browser based GUI for building and running simulations without writing code. It covers geometry setup, materials, sources, monitors and result visualisation. See the GUI documentation for a full walkthrough.

The Python API scales better for parameter sweeps, automation and reproducible workflows. From here, this notebook focuses on Python.

1.4.1 Installing Python

Python does not come pre-installed with the tools needed to run this notebook. We install everything through **Miniconda**, a small installer that bundles Python together with `conda`, the tool that manages packages and environments.

1. Go to docs.conda.io/miniconda and download the installer for your operating system.
2. Run it, accepting the default options.
3. Open a terminal. On Windows this is the **Anaconda Prompt**, installed alongside Miniconda. On macOS or Linux, use the built-in Terminal app.

Run the following once, to create a dedicated environment called `tidy3d`, activate it, and launch Jupyter from inside it:

Before opening this notebook, run the following once in your terminal:

```
# create a dedicated environment with Python inside it
conda create -n tidy3d python=3.10 -y
```

```
# activate it
conda activate tidy3d

# install Jupyter into this environment
pip install jupyterlab

# launch Jupyter from the active environment
jupyter lab
```

Jupyter opens in your browser. Every notebook you open from this tab now runs inside the `tidy3d` environment. Keep this terminal window open, it is running the Jupyter server.

1.4.2 Alternative: Google Colab

No installation is required. Open colab.research.google.com, create a new notebook.

1.4.3 Installing tidy3d

Run the cell below. It installs the tidy3d package into the environment this notebook is using.

[ ]:
```
%pip install tidy3d
```

1.4.4 Checking the version

[ ]:
```
import tidy3d as td

print(td.__version__)
```

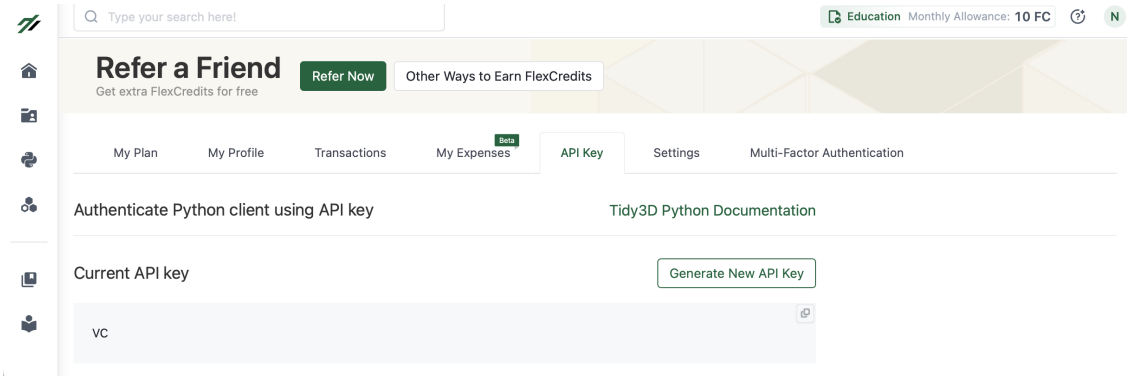
```
2.12.0
```

1.4.5 Getting your API key

Sign in at tidy3d.simulation.cloud, open the Account Center, and go to the API key tab. Copy the key shown there.

The API key identifies you to Flexcompute's servers. Anyone holding it can submit simulations and spend your FlexCredits on your account, so it should be treated the same way as a password.

[ ]:
```
from IPython.display import Image, display

display(Image(filename="/content/images/API.png"))
```



1.4.6 Adding your API key

Run the cell below and paste your API key when prompted, then press Enter.

[ ]:
```
from getpass import getpass
import tidy3d.web as web

api_key = getpass("Paste your tidy3d API key here and press Enter: ")
web.configure(api_key)
```

```
Paste your tidy3d API key here and press Enter: ··········
Configuration saved successfully.
License cache refreshed (0 cached file(s) removed; next local license check will
fetch current entitlements).
```

1.4.7 Verifying the key

[ ]:
```
web.test()
```

```
05:25:36 UTC Authentication configured successfully!
```

1.4.8 Alternative: configuring once for all notebooks

Instead of typing the key every time, you can store it permanently via the terminal.

Windows (Anaconda Prompt):

```
pip install pipx
pipx run tidy3d configure --apikey=YOUR_API_KEY
```

macOS and Linux (Terminal):

```
tidy3d configure --apikey=YOUR_API_KEY
```

This is the same config file that `web.configure()` writes in the notebook cell above. Both methods are interchangeable, one is enough.

1.5 A simple simulation: a silver disk

This section builds a first complete simulation in Tidy3D. A silver disk sits in air and is illuminated by a plane wave with a defined polarisation. The induced field is animated over one oscillation cycle, and the reflected, transmitted and absorbed power are plotted as spectra.

1.6 Workflow

1. Define the topology: the disk geometry and the silver medium.
2. Define the background medium and the boundary conditions.
3. Define the numerical grid.
4. Visualise the setup in 2D and 3D.
5. Estimate the simulation cost.
6. Run the simulation.
7. Visualise the results: the field animation and the T, R, A spectra.

1.6.1 1. Topology

A single silver disk, 200 nm in diameter and 30 nm thick, centered at the origin.

[ ]:
```
%pip install ipywidgets
```

[ ]:
```
import numpy as np
import matplotlib.pyplot as plt

import tidy3d as td

print("Tidy3D version:", td.__version__)
```

```
Tidy3D version: 2.11.2
```

[ ]:
```
# Geometry parameters for the silver disk, entered in nanometres
# and converted to Tidy3D's default unit, microns
NM_TO_UM = 1e-3

disk_diameter_nm = 200.0
disk_thickness_nm = 30.0

disk_radius_um = (disk_diameter_nm / 2) * NM_TO_UM
disk_thickness_um = disk_thickness_nm * NM_TO_UM

print("Disk radius [um]:", disk_radius_um)
print("Disk thickness [um]:", disk_thickness_um)
```

```
Disk radius [um]: 0.1
Disk thickness [um]: 0.03
```

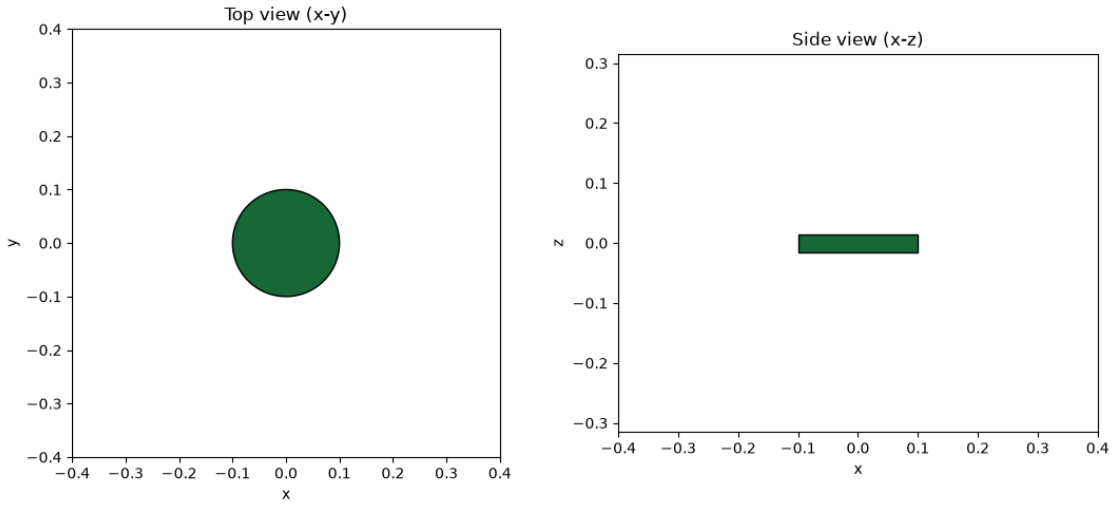
[ ]:
```
# Background medium: air surrounding the disk.
# Required by the Scene object below, separate from
# the boundary conditions that come in the next step.
```

```
air = td.Medium(
    permittivity=1.0,
    name="Air",
)

# Silver, taken from the Tidy3D material library
silver = td.material_library["Ag"]["RakicLorentzDrude1998"]

# The disk itself: a cylinder along z, combined with the silver medium
disk = td.Structure(
    geometry=td.Cylinder(
        center=(0, 0, 0),
        radius=disk_radius_um,
        length=disk_thickness_um,
        axis=2,
    ),
    medium=silver,
    name="Silver disk",
)

print("Disk bounds:", disk.geometry.bounds)
```

```
Disk bounds: ((-0.1, -0.1, -0.015), (0.1, 0.1, 0.015))
```

[ ]:
```
# A Scene bundles structures with a background medium.
# It exists to define and visualize topology,
# before boundary conditions, grid or sources are added.
scene = td.Scene(
    structures=[disk],
    medium=air,
)
```

[ ]:
```
fig, (ax_top, ax_side) = plt.subplots(1, 2, figsize=(11, 5))

# Top view: x-y plane through the middle of the disk
disk.plot(z=0, ax=ax_top)
ax_top.set_title("Top view (x-y)")
ax_top.set_aspect("equal")

# Side view: x-z plane through the centre of the disk
disk.plot(y=0, ax=ax_side)
ax_side.set_title("Side view (x-z)")
ax_side.set_aspect("equal")

plt.tight_layout()
plt.show()
```



[ ]:
```
# Tidy3D's own interactive 3D renderer.
# Opens an inline, rotatable 3D view of the scene.
scene.plot_3d()
```

```
<IPython.core.display.HTML object>
```

1.6.2 3. Numerical grid

The grid resolution balances accuracy and cost. Finer grids improve precision but increase Flex-Credits, as more grid points are updated. This example uses a coarse 4 nm grid for speed and cost-efficiency. A production run would likely use a finer grid, around 1 or 2 nm.

[ ]:
```
# Uniform grid step size, applied across the whole domain.
# A smaller value gives a more accurate result at a higher cost.
# 4 nm is coarse, chosen here for a fast, cheap demonstration run.
grid_cell_nm = 4.0
grid_cell_um = grid_cell_nm * NM_TO_UM

grid_spec = td.GridSpec.uniform(dl=grid_cell_um)

print("Grid cell size [um]:", grid_cell_um)
```

```
Grid cell size [um]: 0.004
```

1.6.3 2. Boundary conditions and light source

The simulation has artificial edges, but light should not reflect from them. Therefore, the top and bottom use PML, or perfectly matched layers. These are artificial absorbing regions that remove outgoing light and imitate open space. Some empty space is left between the disk and the PML so that the absorbing layer does not affect the field near the disk.

A short pulse travels downward onto the disk. It contains wavelengths from 400 to 800 nm, so one

simulation gives the response across the whole range. The light is x-polarised, meaning that its electric field points along the x-axis.

Periodic boundaries are used in x and y, so the disk represents one element of an infinite repeating array.

[ ]:
```
# Spectral range for the incident light, in microns
wavelength_min_um = 0.4
wavelength_max_um = 0.8
num_spectral_points = 101

wavelengths_um = np.linspace(
    wavelength_min_um,
    wavelength_max_um,
    num_spectral_points,
)

# c = f * lambda; td.C_0 is the speed of light in um/s
frequencies_hz = td.C_0 / wavelengths_um

frequency_min_hz = float(frequencies_hz.min())
frequency_max_hz = float(frequencies_hz.max())

source_time = td.GaussianPulse.from_frequency_range(
    fmin=frequency_min_hz,
    fmax=frequency_max_hz,
)

central_wavelength_um = td.C_0 / source_time.freq0

print("Wavelength range [um]:", wavelength_min_um, wavelength_max_um)
print("Central wavelength [um]:", central_wavelength_um)
```

```
Wavelength range [um]: 0.4 0.8
Central wavelength [um]: 0.5953904465101941
```

[ ]:
```
# Numerical clearance between the disk and each PML along z
pml_clearance_factor = 0.6
pml_clearance_um = pml_clearance_factor * central_wavelength_um

z_disk_top = disk_thickness_um / 2
z_disk_bottom = -disk_thickness_um / 2

z_max = z_disk_top + pml_clearance_um
z_min = z_disk_bottom - pml_clearance_um

# Place the source above the disk, halfway to the upper PML
source_position_fraction = 0.5
```

```
z_source = z_disk_top + source_position_fraction * pml_clearance_um

simulation_size_z = z_max - z_min
simulation_center_z = (z_max + z_min) / 2

# The disk is one cell of a periodic array. The cell is wider than
# the disk, so neighbouring disks have a gap between them.
period_nm = 400.0
period_um = period_nm * NM_TO_UM
simulation_size_xy = period_um

print("Period [um]:", period_um)
print("PML clearance [um]:", pml_clearance_um)
print("Source z [um]:", z_source)
print(
    "Simulation size [um]:",
    simulation_size_xy,
    simulation_size_xy,
    simulation_size_z,
)
```

```
Period [um]: 0.4
PML clearance [um]: 0.35723426790611645
Source z [um]: 0.19361713395305824
Simulation size [um]: 0.4 0.4 0.7444685358122329
```

[ ]:
```
# A plane wave is the right source here: with periodic boundaries in
# x and y it is consistent with the walls, and it illuminates every
# disk in the array identically.
plane_wave = td.PlaneWave(
    center=(0, 0, z_source),
    size=(td.inf, td.inf, 0),
    source_time=source_time,
    direction="-",
    angle_theta=0,
    pol_angle=0,
    name="Normal incidence plane wave",
)

# Periodic in x and y: the disk repeats sideways as an array.
# Absorbing (PML) only along z, where light enters and leaves.
boundary_spec = td.BoundarySpec(
    x=td.Boundary.periodic(),
    y=td.Boundary.periodic(),
    z=td.Boundary.pml(),
)
```

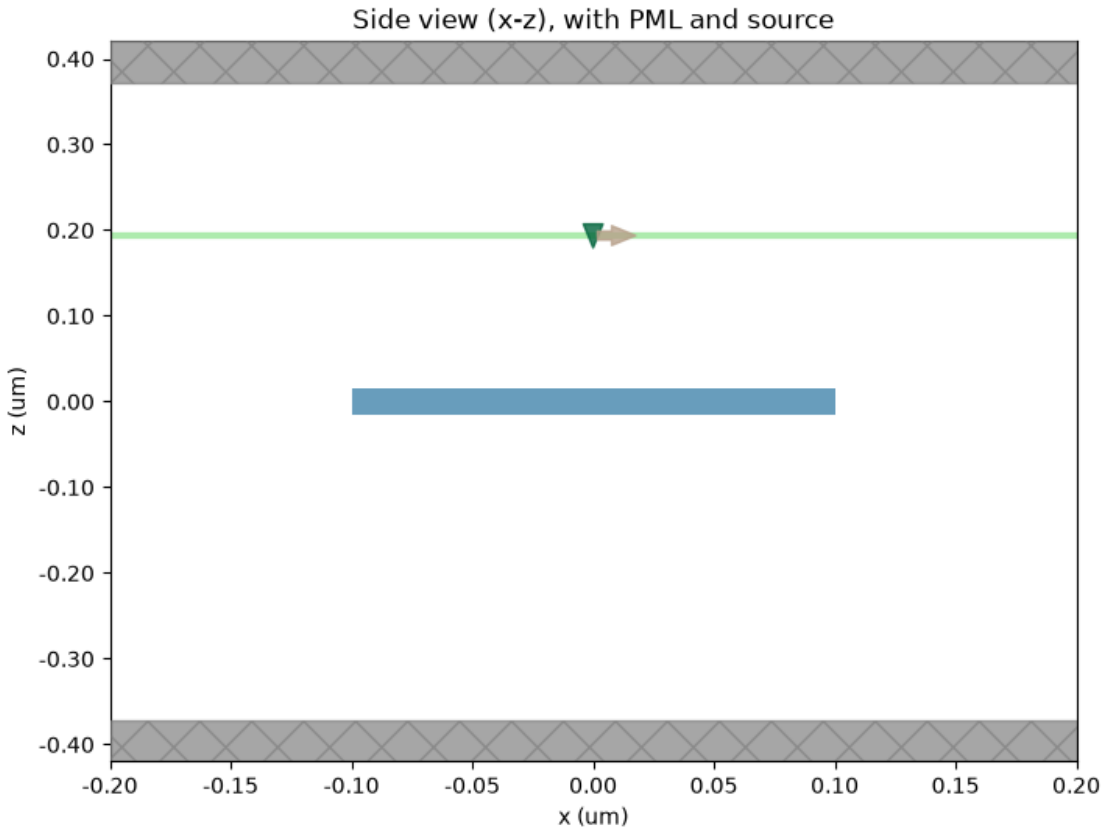
```
sim = td.Simulation(
    center=(0, 0, simulation_center_z),
    size=(simulation_size_xy, simulation_size_xy, simulation_size_z),
    medium=air,
    structures=[disk],
    sources=[plane_wave],
    boundary_spec=boundary_spec,
    grid_spec=grid_spec,
    # Placeholder; the run time is set properly once monitors are added
    run_time=1e-14,
)

print("Simulation size [um]:", sim.size)
```

```
Simulation size [um]: (0.4, 0.4, 0.7444685358122329)
```

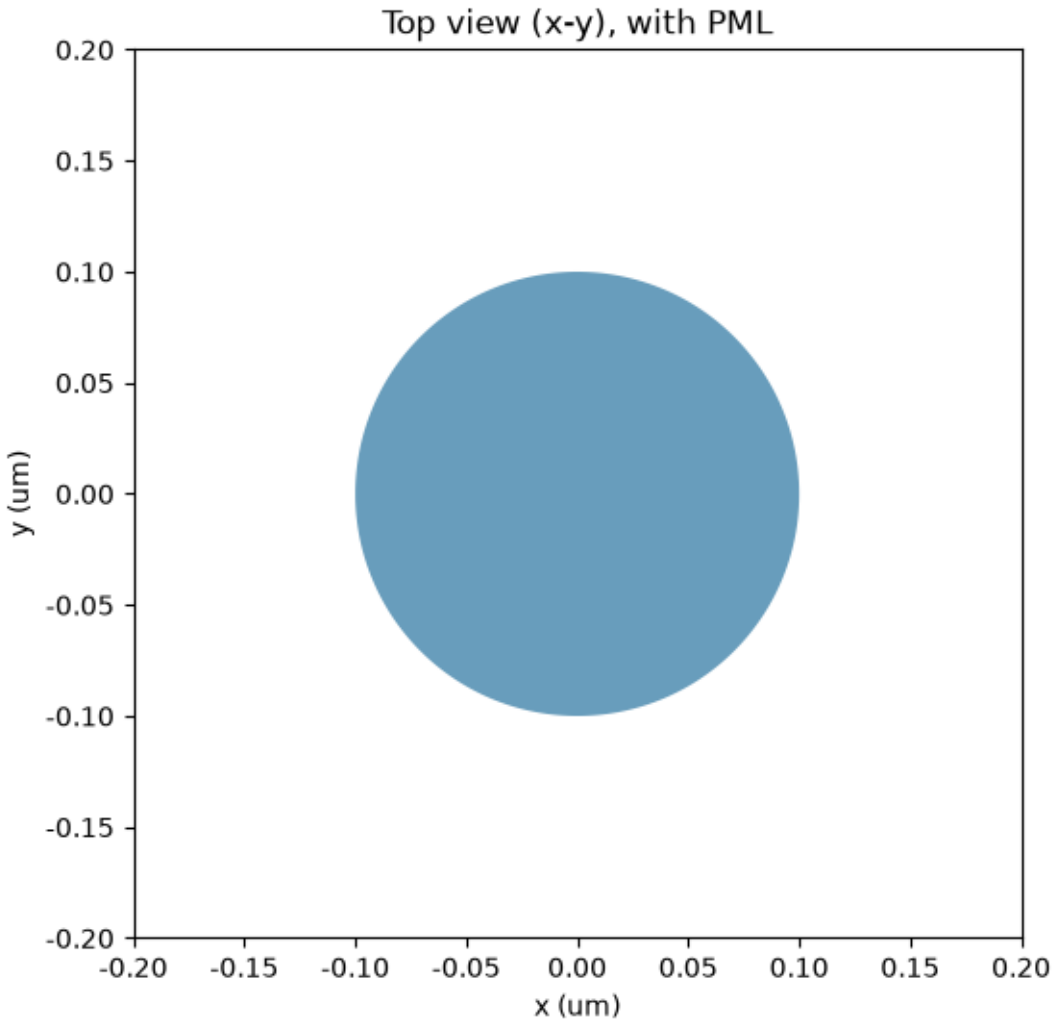
1.6.4 Checking the setup

Two views of the box: from the side, and from above. The shaded band around the edges is the PML, the part that absorbs light. The dashed red line is the light source, with an arrow showing which way it points.

[ ]:
```
fig, ax = plt.subplots(figsize=(8, 6))

sim.plot(y=0, ax=ax)

ax.set_title("Side view (x-z), with PML and source")
ax.set_xlabel("x (um)")
ax.set_ylabel("z (um)")
ax.set_aspect("auto")

plt.show()
```



[ ]:
```
fig, ax = plt.subplots(figsize=(6, 6))

sim.plot(z=0, ax=ax)

ax.set_title("Top view (x-y), with PML")
ax.set_xlabel("x (um)")
ax.set_ylabel("y (um)")
ax.set_aspect("equal")

plt.show()
```



1.6.5 Recording the field over time

To animate how charge moves on the disk, the simulation needs a monitor that saves snapshots of the field while it runs, not just the final result. This is called a `FieldTimeMonitor`.

It sits in a small box wrapped around the disk, a bit bigger than the disk on every side, so it also catches the field just outside the disk's surfaces. That extra margin is what lets us work out later where charge is piling up, from how the field jumps right at the disk's edges.

Saving a snapshot every single time step would be far more data than needed, so this monitor only saves one every 10 steps.

[ ]:
```
# Small 3D box around the disk, padded a bit beyond its edges,
# so the field just outside each surface is captured too.
field_padding_um = 0.5 * disk_radius_um
```

```
field_time_monitor = td.FieldTimeMonitor(
    center=(0, 0, 0),
    size=(
        2 * (disk_radius_um + field_padding_um),
        2 * (disk_radius_um + field_padding_um),
        disk_thickness_um + 2 * field_padding_um,
    ),
    interval=10,
    name="disk_field_time",
)

sim = sim.updated_copy(
    monitors=[field_time_monitor],
)

print("Field monitor size [um]:", field_time_monitor.size)
```

```
Field monitor size [um]: (0.30000000000000004, 0.30000000000000004, 0.13)
```

[ ]:
```
# Reflection monitor: halfway between the source and the upper PML
z_reflection_monitor = 0.5 * (z_source + z_max)

# Transmission monitor: halfway between the disk's bottom and the lower PML
z_transmission_monitor = 0.5 * (z_disk_bottom + z_min)

reflection_monitor = td.FluxMonitor(
    center=(0, 0, z_reflection_monitor),
    size=(td.inf, td.inf, 0),
    freqs=frequencies_hz,
    normal_dir="+",
    name="reflection",
)

transmission_monitor = td.FluxMonitor(
    center=(0, 0, z_transmission_monitor),
    size=(td.inf, td.inf, 0),
    freqs=frequencies_hz,
    normal_dir="-",
    name="transmission",
)

sim = sim.updated_copy(
    monitors=[
        reflection_monitor,
        transmission_monitor,
        field_time_monitor,
    ],
```

```
)

print("Reflection monitor z [um]:", z_reflection_monitor)
print("Transmission monitor z [um]:", z_transmission_monitor)
print("Monitors:", [m.name for m in sim.monitors])
```

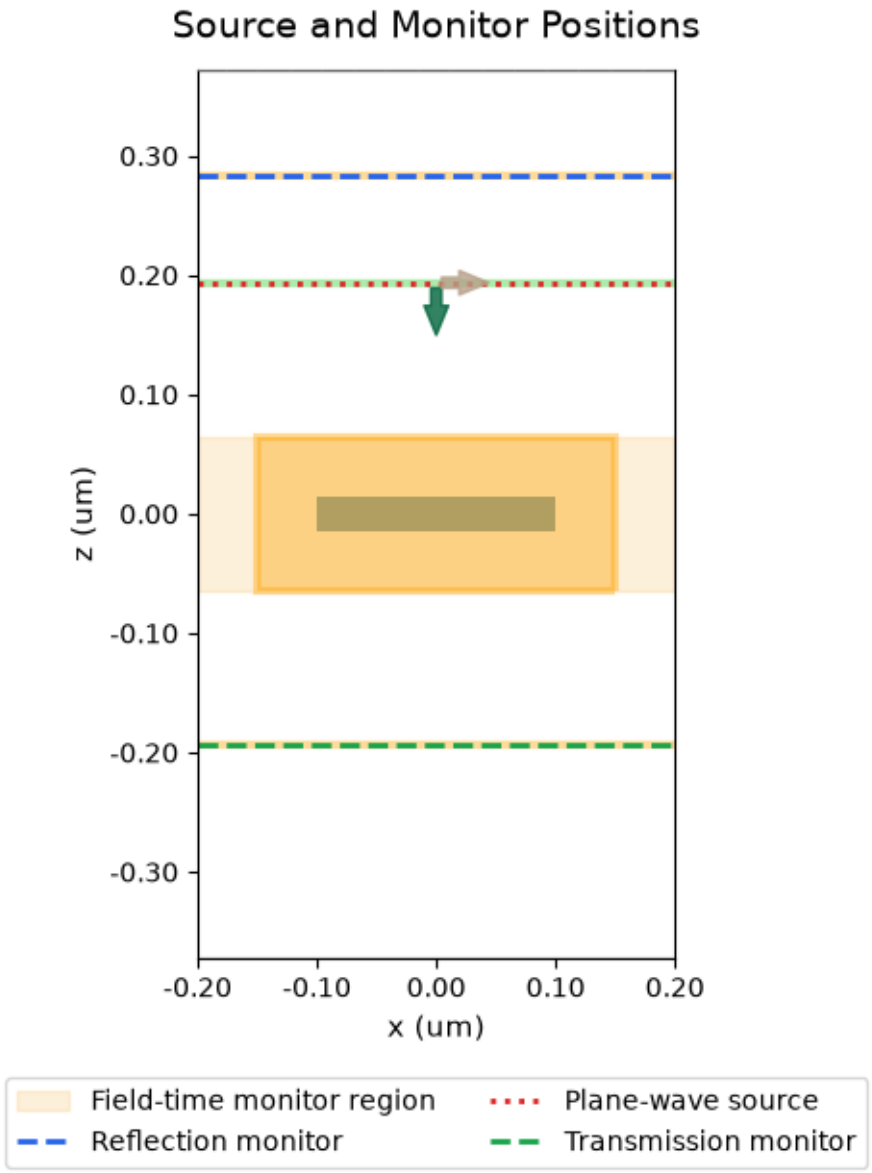
```
Reflection monitor z [um]: 0.28292570092958735
Transmission monitor z [um]: -0.19361713395305824
Monitors: ['reflection', 'transmission', 'disk_field_time']
```

1.6.6 Source and monitor positions

The plot below marks where the source sits, where each flux monitor sits, and the region covered by the field-time monitor, all on one cross section.

[ ]:
```
fig, ax = plt.subplots(figsize=(9, 7))

sim.plot(y=0, ax=ax)

field_region = ax.axhspan(
    -field_time_monitor.size[2] / 2,
    field_time_monitor.size[2] / 2,
    color="#F59E0B",
    alpha=0.15,
    label="Field-time monitor region",
)

reflection_line = ax.axhline(
    z_reflection_monitor,
    color="#2563EB",
    linestyle="--",
    linewidth=2,
    label="Reflection monitor",
)

source_line = ax.axhline(
    z_source,
    color="#DC2626",
    linestyle=":",
    linewidth=2,
    label="Plane-wave source",
)

transmission_line = ax.axhline(
    z_transmission_monitor,
    color="#16A34A",
    linestyle="--",
    linewidth=2,
```

```
    label="Transmission monitor",
)

ax.set_title("Source and Monitor Positions", fontsize=14, pad=12)
ax.set_xlabel("x (um)", fontsize=11)
ax.set_ylabel("z (um)", fontsize=11)
ax.set_ylim(z_min, z_max)

ax.legend(
    handles=[field_region, reflection_line, source_line, transmission_line],
    loc="upper center",
    bbox_to_anchor=(0.5, -0.12),
    ncol=2,
    frameon=True,
    fontsize=10,
)

fig.subplots_adjust(bottom=0.22)

plt.show()
```



1.6.7 Estimating the cost

Before running the simulation for real, Tidy3D can estimate its maximum cost in FlexCredits. This uploads the setup to the cloud but does not run it, so it does not use up any credits.

First, the simulation needs a real run time. So far it only had a placeholder value.

[ ]:
```
# A generous upper bound on how long the simulation runs.
# In most cases the field inside and around the disk decays well
# before this point, and Tidy3D's default shutoff (it stops early
# once the remaining field energy drops to 1e-5 of its peak) ends
# the run automatically. This mainly protects against a slow-decaying
```

```
# case that needs more time than expected.
run_time_fs = 50.0
FS_TO_S = 1e-15

sim = sim.updated_copy(
    run_time=run_time_fs * FS_TO_S,
)

print("Run time [fs]:", run_time_fs)
```

```
Run time [fs]: 50.0
```

[ ]:
```
import tidy3d.web as web

# Uploading the task and estimating its cost does not run the
# simulation and does not spend any FlexCredits.
cost_check_job = web.Job(
    simulation=sim,
    task_name="silver_disk_cost_check",
    verbose=True,
)

estimated_cost = web.estimate_cost(cost_check_job.task_id)

print("Estimated maximum cost [FlexCredits]:", estimated_cost)
```

```
↑ simulation.hdf5.gz  100.0% • 3.0/3.0 kB • ? • 0:00:00
```

```
12:44:05 CST Estimated FlexCredit cost: 0.025. Minimum cost depends on task
             execution details. Use 'web.real_cost(task_id)' to get the billed
             FlexCredit cost after a simulation run.
```

```
Estimated maximum cost [FlexCredits]: 0.025
```

1.6.8 Running the simulation

0.025 FlexCredits is a very small price, a tiny fraction of even one trial credit, so this is safe to run.

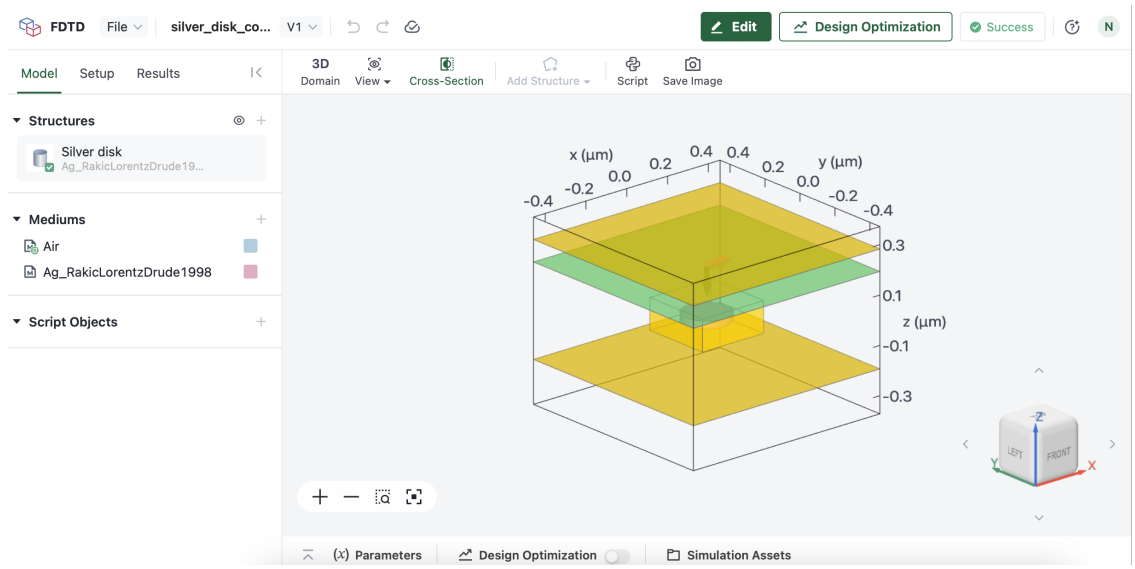
The same task that was already uploaded for the cost check can now be run directly.

[ ]:
```
retry_job = web.Job(
    simulation=sim,
    task_name="silver_disk_run_2",
    verbose=True,
)
```

```
sim_data = retry_job.run(
    path="silver_disk_data.hdf5",
)

print("Simulation complete.")
print("Available monitors:", [m.name for m in sim_data.simulation.monitors])
```

That link opens Tidy3D's web dashboard, with more detail on this run: logs, a 3D viewer, and other diagnostics. This notebook keeps working with the same result directly, no need to go to that page for what comes next.

[ ]:
```
from IPython.display import Image, display

display(Image(filename="/content/images/results.png"))
```



1.6.9 Getting the data back out

The result of this run is now stored on Tidy3D's servers, under a task ID. That ID can be used later, even in a brand new session, to fetch the exact same result again, without paying for or running the simulation a second time.

[ ]:
```
task_id = retry_job.task_id

print("Task ID:", task_id)
```

To reload it later, this one call is all that is needed. It downloads the result if the local file is missing, or reads it straight from disk if it is already there.

[ ]:
```
# Load the data through task-id
sim_data = web.load('fdve-f7c6108a-257d-40fd-8354-88407000c77d')
```

```
Output()
```

```
05:27:13 UTC Loading results from simulation_data.hdf5
```

```
05:27:17 UTC WARNING: updating Simulation from 2.11 to 2.12
```

```
WARNING: Simulation final field decay value of 8.23e-05 is greater
than the simulation shutoff threshold of 1e-05. Consider running
the simulation again with a larger 'run_time' duration for more
accurate results.
```

1.6.10 Reflection, transmission and absorption

The two flux monitors give reflection and transmission as a function of frequency. Absorption is worked out from them, as A = 1 - R - T.

[ ]:
```
import numpy as np

reflection_flux = sim_data["reflection"].flux
transmission_flux = sim_data["transmission"].flux

result_frequencies_hz = np.asarray(reflection_flux.coords["f"].values)
result_wavelengths_um = td.C_0 / result_frequencies_hz

R = np.asarray(reflection_flux.values, dtype=float).squeeze()
T = np.asarray(transmission_flux.values, dtype=float).squeeze()
A = 1 - R - T

print("Number of frequency points:", len(result_frequencies_hz))
print("R range:", R.min(), R.max())
print("T range:", T.min(), T.max())
print("A range:", A.min(), A.max())
```

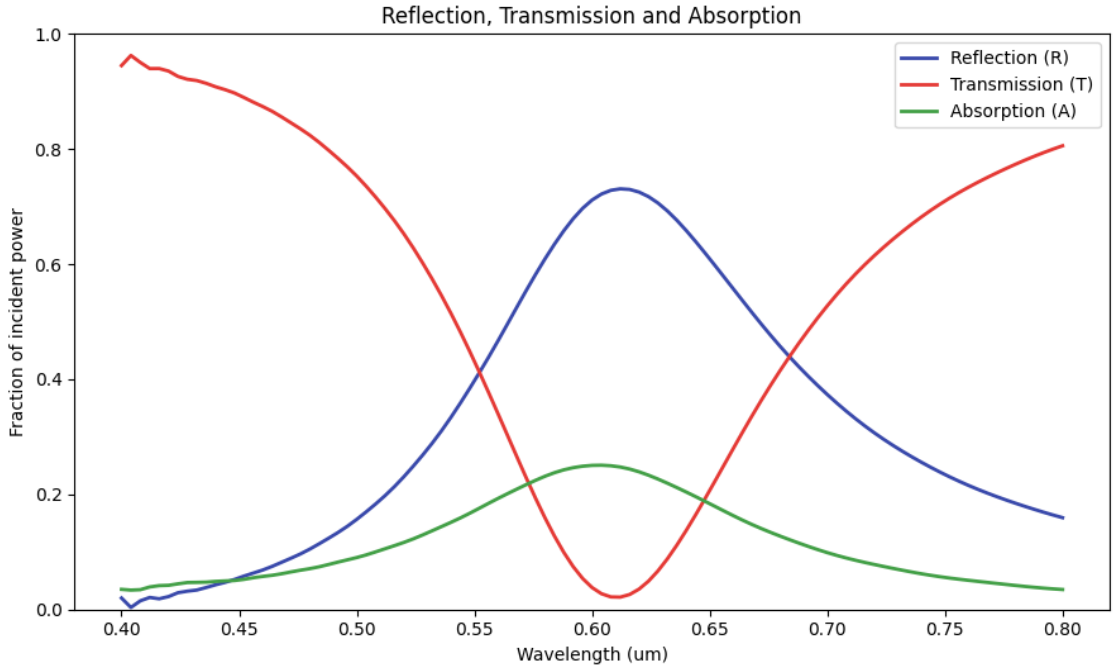
```
Number of frequency points: 101
R range: 0.003568763844668865 0.7309263944625854
T range: 0.021488670259714127 0.9626717567443848
A range: 0.03375947941094637 0.2506920639425516
```

1.6.11 What R, T and A mean

When light hits the disk, three things can happen to it. Some bounces back, this is reflection (R). Some passes straight through, this is transmission (T). The rest is absorbed by the silver and turned into heat, this is absorption (A). Together they always add up to the whole incoming power, so R + T + A = 1 at every wavelength.

R and T come straight from the two flux monitors, one number per wavelength each. A is not measured by any monitor at all. It is worked out afterwards, using A = 1 - R - T.

[ ]:
```
import matplotlib.pyplot as plt

# Plot the results

fig, ax = plt.subplots(figsize=(9, 5.5))

ax.plot(result_wavelengths_um, R, color="#3949AB", linewidth=2,␣
↪label="Reflection (R)")
ax.plot(result_wavelengths_um, T, color="#E53935", linewidth=2,␣
↪label="Transmission (T)")
ax.plot(result_wavelengths_um, A, color="#43A047", linewidth=2,␣
↪label="Absorption (A)")

ax.set_xlabel("Wavelength (um)")
ax.set_ylabel("Fraction of incident power")
ax.set_title("Reflection, Transmission and Absorption")
ax.set_ylim(0, 1)
ax.legend()

plt.tight_layout()
plt.show()
```



[ ]:
```
import numpy as np
import xarray as xr
import plotly.graph_objects as go

# ------------------------------------------------------------
# 1. Load the electric field recorded by the FieldTimeMonitor
# ------------------------------------------------------------
field = sim_data["disk_field_time"]

Ex = field.Ex
Ey = field.Ey
Ez = field.Ez

times = np.asarray(Ex.coords["t"].values)

# Limit the number of animation frames to keep Plotly responsive
frame_indices = np.linspace(
    0,
    len(times) - 1,
    min(60, len(times)),
    dtype=int,
)

# Distance used to sample the field just outside and inside the metal
delta_um = grid_cell_um


# ------------------------------------------------------------
# 2. Build a triangular mesh of the complete disk surface
# ------------------------------------------------------------
n_phi = 72
n_z = 12
n_r = 12

phi = np.linspace(0, 2 * np.pi, n_phi, endpoint=False)

vertices_x = []
vertices_y = []
vertices_z = []

normals_x = []
normals_y = []
normals_z = []

triangles_i = []
triangles_j = []
triangles_k = []
```

```
def add_vertex(x, y, z, nx, ny, nz):
    """Add one mesh vertex and return its index."""

    index = len(vertices_x)

    vertices_x.append(x)
    vertices_y.append(y)
    vertices_z.append(z)

    normals_x.append(nx)
    normals_y.append(ny)
    normals_z.append(nz)

    return index


# ------------------------------------------------------------
# 2a. Cylindrical side surface
# ------------------------------------------------------------
side_indices = np.empty((n_z, n_phi), dtype=int)

z_values = np.linspace(
    -disk_thickness_um / 2,
    disk_thickness_um / 2,
    n_z,
)

for iz, z_value in enumerate(z_values):
    for iphi, phi_value in enumerate(phi):
        cos_phi = np.cos(phi_value)
        sin_phi = np.sin(phi_value)

        side_indices[iz, iphi] = add_vertex(
            disk_radius_um * cos_phi,
            disk_radius_um * sin_phi,
            z_value,
            cos_phi,
            sin_phi,
            0.0,
        )

# Connect neighbouring side vertices with triangles
for iz in range(n_z - 1):
    for iphi in range(n_phi):
        next_phi = (iphi + 1) % n_phi
```

```
        v00 = side_indices[iz, iphi]
        v01 = side_indices[iz, next_phi]
        v10 = side_indices[iz + 1, iphi]
        v11 = side_indices[iz + 1, next_phi]

        triangles_i.extend([v00, v00])
        triangles_j.extend([v10, v11])
        triangles_k.extend([v11, v01])


# ------------------------------------------------------------
# 2b. Top circular surface
# ------------------------------------------------------------
top_center = add_vertex(
    0.0,
    0.0,
    disk_thickness_um / 2,
    0.0,
    0.0,
    1.0,
)

top_indices = np.empty((n_r, n_phi), dtype=int)

radial_values = np.linspace(
    disk_radius_um / n_r,
    disk_radius_um,
    n_r,
)

for ir, radius in enumerate(radial_values):
    for iphi, phi_value in enumerate(phi):
        top_indices[ir, iphi] = add_vertex(
            radius * np.cos(phi_value),
            radius * np.sin(phi_value),
            disk_thickness_um / 2,
            0.0,
            0.0,
            1.0,
        )

# Connect the centre to the first radial ring
for iphi in range(n_phi):
    next_phi = (iphi + 1) % n_phi

    triangles_i.append(top_center)
```

```
    triangles_j.append(top_indices[0, iphi])
    triangles_k.append(top_indices[0, next_phi])

# Connect neighbouring radial rings
for ir in range(n_r - 1):
    for iphi in range(n_phi):
        next_phi = (iphi + 1) % n_phi

        v00 = top_indices[ir, iphi]
        v01 = top_indices[ir, next_phi]
        v10 = top_indices[ir + 1, iphi]
        v11 = top_indices[ir + 1, next_phi]

        triangles_i.extend([v00, v00])
        triangles_j.extend([v11, v10])
        triangles_k.extend([v01, v11])


# ------------------------------------------------------------
# 2c. Bottom circular surface
# ------------------------------------------------------------
bottom_center = add_vertex(
    0.0,
    0.0,
    -disk_thickness_um / 2,
    0.0,
    0.0,
    -1.0,
)

bottom_indices = np.empty((n_r, n_phi), dtype=int)

for ir, radius in enumerate(radial_values):
    for iphi, phi_value in enumerate(phi):
        bottom_indices[ir, iphi] = add_vertex(
            radius * np.cos(phi_value),
            radius * np.sin(phi_value),
            -disk_thickness_um / 2,
            0.0,
            0.0,
            -1.0,
        )

# Connect the centre to the first radial ring
for iphi in range(n_phi):
    next_phi = (iphi + 1) % n_phi
```

```
    triangles_i.append(bottom_center)
    triangles_j.append(bottom_indices[0, next_phi])
    triangles_k.append(bottom_indices[0, iphi])

# Connect neighbouring radial rings
for ir in range(n_r - 1):
    for iphi in range(n_phi):
        next_phi = (iphi + 1) % n_phi

        v00 = bottom_indices[ir, iphi]
        v01 = bottom_indices[ir, next_phi]
        v10 = bottom_indices[ir + 1, iphi]
        v11 = bottom_indices[ir + 1, next_phi]

        triangles_i.extend([v00, v00])
        triangles_j.extend([v10, v11])
        triangles_k.extend([v11, v01])


# Convert mesh data to NumPy arrays
x = np.asarray(vertices_x)
y = np.asarray(vertices_y)
z = np.asarray(vertices_z)

nx = np.asarray(normals_x)
ny = np.asarray(normals_y)
nz = np.asarray(normals_z)

triangles_i = np.asarray(triangles_i)
triangles_j = np.asarray(triangles_j)
triangles_k = np.asarray(triangles_k)


# ------------------------------------------------------------
# 3. Create sampling points outside and inside the disk
# ------------------------------------------------------------
x_out = x + delta_um * nx
y_out = y + delta_um * ny
z_out = z + delta_um * nz

x_in = x - delta_um * nx
y_in = y - delta_um * ny
z_in = z - delta_um * nz


def sample_field(component, xq, yq, zq):
    """Interpolate one electric-field component at arbitrary 3D points."""
```

```
    sampled = component.interp(
        x=xr.DataArray(xq, dims="point"),
        y=xr.DataArray(yq, dims="point"),
        z=xr.DataArray(zq, dims="point"),
    )

    sampled = sampled.transpose("point", "t")

    return np.real(sampled.values[:, frame_indices])


# ------------------------------------------------------------
# 4. Sample the electric field outside and inside the metal
# ------------------------------------------------------------
Ex_out = sample_field(Ex, x_out, y_out, z_out)
Ey_out = sample_field(Ey, x_out, y_out, z_out)
Ez_out = sample_field(Ez, x_out, y_out, z_out)

Ex_in = sample_field(Ex, x_in, y_in, z_in)
Ey_in = sample_field(Ey, x_in, y_in, z_in)
Ez_in = sample_field(Ez, x_in, y_in, z_in)


# ------------------------------------------------------------
# 5. Calculate the surface charge density
#
# sigma = epsilon_0 * (E_out - E_in) · n
# ------------------------------------------------------------
epsilon_0 = 8.8541878128e-12

sigma = epsilon_0 * (
    (Ex_out - Ex_in) * nx[:, None]
    + (Ey_out - Ey_in) * ny[:, None]
    + (Ez_out - Ez_in) * nz[:, None]
)

# Replace interpolation failures with zero
sigma = np.nan_to_num(
    sigma,
    nan=0.0,
    posinf=0.0,
    neginf=0.0,
)

# Use one fixed colour scale for the entire animation
color_limit = np.percentile(np.abs(sigma), 99)
```

```
if not np.isfinite(color_limit) or color_limit == 0:
    color_limit = 1.0

frame_times_fs = times[frame_indices] * 1e15


# ------------------------------------------------------------
# 6. Create the initial continuous surface
# ------------------------------------------------------------
initial_surface = go.Mesh3d(
    x=x,
    y=y,
    z=z,
    i=triangles_i,
    j=triangles_j,
    k=triangles_k,
    intensity=sigma[:, 0],
    intensitymode="vertex",
    colorscale="RdBu_r",
    cmin=-color_limit,
    cmax=color_limit,
    flatshading=False,
    opacity=1.0,
    showscale=True,
    colorbar=dict(
        title="σ<br>C/m²",
        thickness=18,
    ),
    lighting=dict(
        ambient=0.75,
        diffuse=0.65,
        specular=0.15,
        roughness=0.8,
        fresnel=0.05,
    ),
    lightposition=dict(
        x=2,
        y=2,
        z=3,
    ),
    hovertemplate=(
        "x = %{x:.4f} µm<br>"
        "y = %{y:.4f} µm<br>"
        "z = %{z:.4f} µm<br>"
        "σ = %{intensity:.3e} C/m²"
        "<extra></extra>"
```

```
    ),
)


# ------------------------------------------------------------
# 7. Create the animation frames
# ------------------------------------------------------------
frames = []

for frame_number, time_fs in enumerate(frame_times_fs):
    frames.append(
        go.Frame(
            name=str(frame_number),
            data=[
                go.Mesh3d(
                    x=x,
                    y=y,
                    z=z,
                    i=triangles_i,
                    j=triangles_j,
                    k=triangles_k,
                    intensity=sigma[:, frame_number],
                    intensitymode="vertex",
                    colorscale="RdBu_r",
                    cmin=-color_limit,
                    cmax=color_limit,
                    flatshading=False,
                    opacity=1.0,
                )
            ],
            traces=[0],
            layout=go.Layout(
                title_text=(
                    "Surface charge on the silver disk"
                    f" - t = {time_fs:.2f} fs"
                )
            ),
        )
    )


# ------------------------------------------------------------
# 8. Display the interactive 3D animation
# ------------------------------------------------------------
fig = go.Figure(
    data=[initial_surface],
    frames=frames,
```

```
)
frame_duration_ms = 400

fig.update_layout(
    title=(
        "Surface charge on the silver disk"
        f" - t = {frame_times_fs[0]:.2f} fs"
    ),
    scene=dict(
        xaxis_title="x (µm)",
        yaxis_title="y (µm)",
        zaxis_title="z (µm)",
        aspectmode="data",
        camera=dict(
            eye=dict(
                x=1.5,
                y=1.5,
                z=1.0,
            )
        ),
    ),
    width=850,
    height=700,
    margin=dict(
        l=0,
        r=0,
        b=0,
        t=70,
    ),
    updatemenus=[
        dict(
            type="buttons",
            showactive=False,
            x=0.0,
            y=1.08,
            buttons=[
                dict(
                    label=" Play",
                    method="animate",
                    args=[
                        None,
                        dict(
                            frame=dict(
                                duration=frame_duration_ms,
                                redraw=True,
                            ),
                            transition=dict(duration=0),
```

```
                        fromcurrent=True,
                        mode="immediate",
                    ),
                ],
            ),
            dict(
                label=" Pause",
                method="animate",
                args=[
                    [None],
                    dict(
                        frame=dict(
                            duration=0,
                            redraw=False,
                        ),
                        transition=dict(duration=0),
                        mode="immediate",
                    ),
                ],
            ),
        ],
    )
],
sliders=[
    dict(
        active=0,
        currentvalue=dict(
            prefix="Time: ",
            suffix=" fs",
        ),
        pad=dict(t=40),
        steps=[
            dict(
                label=f"{time_fs:.2f}",
                method="animate",
                args=[
                    [str(frame_number)],
                    dict(
                        mode="immediate",
                        frame=dict(
                            duration=0,
                            redraw=True,
                        ),
                        transition=dict(duration=0),
                    ),
                ],
            )
```

```
                for frame_number, time_fs in enumerate(frame_times_fs)
            ],
        )
    ],
)

fig.show()
```